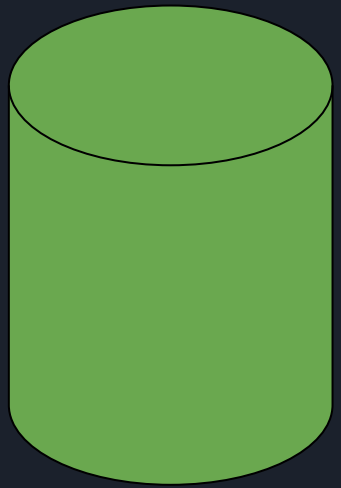
GESTIÓN DE DATOS

REPASO Y "RECORDERIS" DE MODELOS DE DATOS

¿QUÉ RECORDAREMOS HOY?



- Conceptos de bases de datos
- DDL, DML, SQL
- Funciones nativas
- PL/SQL
- etc…

¿Alguien que quiera recordar conceptos?



Sección:
Conceptos

Base de Datos: QUÉ ES

???????



Base de Datos: QUÉ ES



Según Wikipedia, "Una base de datos es un conjunto de datos pertenecientes a un mismo contexto y almacenados sistemáticamente para su posterior uso. En este sentido; una biblioteca puede considerarse una base de datos compuesta en su mayoría por documentos y textos impresos en papel e indexados para su consulta."

Concepto: Persistencia

???????

Concepto: Persistencia

Mecanismo usado para que los datos perduren en el tiempo

Concepto: Concurrencia

???????

Concepto: Concurrencia

Capacidad de un sistema para permitir varias conexiones y operaciones simultáneamente sin afectar su comportamiento normal

Concepto: Integridad

???????

Concepto: Integridad

Reglas, restricciones y condiciones que aseguran la consistencia de los datos en una base de datos

Concepto: Tipos de relaciones

??????? (discusión verbal)

Concepto: Integridad Referencial

???????

Concepto: Integridad Referencial

Garantizar que la relación entre dos tablas exista al momento de insertar, actualizar o eliminar datos. En otras palabras, en una relación 1:n, la tabla referenciada (n) debe tener un registro asociado en la tabla origen (1)

Concepto: Normalización

???????

Concepto: Normalización

Proceso mediante el cual se refina un modelo relacional con el objetivo de disminuir los datos redundantes en la base de datos

Concepto: Base de datos sql

(...) o bases de datos relacionales; se usan generalmente para guardar datos *estructurados* (generalmente en formato tabular) y que tienen *relaciones* entre ellos

Concepto: Base de datos no-sql

(...) o bases de datos no relacionales (Not Only SQL); se usan generalmente para guardar datos *no estructurados* (generalmente en formato documental, llave-valor o grafos) y que pueden o no tener *relaciones* entre ellos

Concepto: DBMS

???????

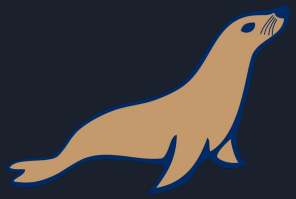
Concepto: DBMS

Sistema Gestor de Bases de Datos (por sus siglas en inglés), es quien se encarga de gestionar / administrar una base de datos

Pregunta: Nombres de DBMS

???????

Pregunta: Nombres de DBMS












Concepto: DBA

???????

Concepto: DBA

Database Administrator (por sus siglas en inglés); persona encargada de administrar y mantener el DBMS funcionando correctamente

Concepto: Clave primaria

??????? (discusión verbal)

Concepto: Clave foránea

??????? (discusión verbal)

Concepto: Tipos de datos

??????? (discusión verbal)

Concepto: M.E.R.

El modelo entidad relación es…

??????? (discusión verbal)

Concepto: M.R.

El modelo relacional es…

??????? (discusión verbal)

Sección:
DDL y DML

Ejemplo 1: algo simple

M.R.

CUSTOMERS	
ID	VARCHAR(20)
ID_TYPE	VARCHAR(3)
FULL_NAME	VARCHAR(100)

SQL

```
create table customers(
    id          varchar(20),
    id_type     varchar(3),
    full_name   varchar(100)
);
```

Ejemplo 2: con clave primaria y obligatorios / opcionales

M.R.

CUSTOMERS		
PK	ID	VARCHAR(20)
*	ID_TYPE	VARCHAR(3)
*	FULL_NAME	VARCHAR(100)

SQL

```
create table customers(
    id          varchar(20)  not null,
    id_type     varchar(3)   not null,
    full_name   varchar(100) not null,
    primary key(id)
);
```

Ejemplo 3: clave primaria compuesta

M.R.

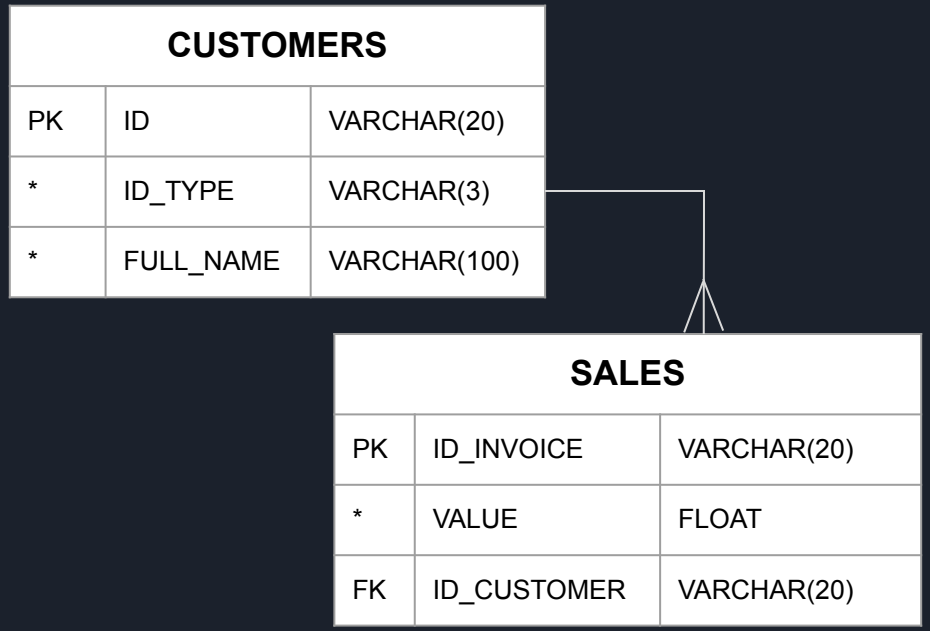
CUSTOMERS		
PK	ID	VARCHAR(20)
PK	ID_TYPE	VARCHAR(3)
*	FULL_NAME	VARCHAR(100)

SQL

```
create table customers(
    id          varchar(20)  not null,
    id_type     varchar(3)   not null,
    full_name   varchar(100) not null,
    primary key(id, id_type)
);
```

Ejemplo 4: con clave foránea

M.R.



SQL

```
create table customers(
    id          varchar(20)  not null,
    id_type     varchar(3)   not null,
    full_name   varchar(100) not null,
    primary key (id)
);

create table sales(
    id_invoice  varchar(20)  not null,
    value       float        not null,
    id_customer varchar(20)  not null,
    primary key (id_invoice),
    constraint  foreign key (id_customer)
    references  customers(id)
);
```

Ejemplo 5: Consultar lo existente

```
show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sakila             |
| sys                |
| world              |
+--------------------+
6 rows in set (0,012 sec)

show tables;
+-----------------+
| Tables_in_world |
+-----------------+
| city            |
| country         |
| countrylanguage |
+-----------------+
3 rows in set (0.00 sec)
```

```
desc customers;
+-----------+--------------+------+-----+---------+-------+
| Field     | Type         | Null | Key | Default | Extra |
+-----------+--------------+------+-----+---------+-------+
| id        | varchar(20)  | NO   | PRI | NULL    |       |
| id_type   | varchar(3)   | NO   |     | NULL    |       |
| full_name | varchar(100) | NO   |     | NULL    |       |
+-----------+--------------+------+-----+---------+-------+
3 rows in set (0.03 sec)

desc sales;
+-------------+-------------+------+-----+---------+-------+
| Field       | Type        | Null | Key | Default | Extra |
+-------------+-------------+------+-----+---------+-------+
| id_invoice  | varchar(20) | NO   | PRI | NULL    |       |
| value       | float       | NO   |     | NULL    |       |
| id_customer | varchar(20) | NO   | MUL | NULL    |       |
+-------------+-------------+------+-----+---------+-------+
3 rows in set (0.00 sec)
```

Ejemplo 6: Modificar una tabla (+)

M.R.

CUSTOMERS		
PK	ID	VARCHAR(20)
PK	ID_TYPE	VARCHAR(3)
*	FULL_NAME	VARCHAR(100)
*	ADDRESS	VARCHAR(100)

SQL

```
alter table customers
add column address varchar(100) not null ;

desc customers; (...)
```

Ejemplo 7: Modificar una tabla (-)

M.R.

CUSTOMERS		
PK	ID	VARCHAR(20)
PK	ID_TYPE	VARCHAR(3)
*	ADDRESS	VARCHAR(100)

SQL

```
alter table customers
drop column full_name ;

desc customers; (...)
```

Ejemplo 8: Modificar una tabla (+ / -)

M.R.

CUSTOMERS		
PK	ID	VARCHAR(20)
PK	ID_TYPE	VARCHAR(3)
*	FULL_NAME	VARCHAR(100)

SQL

```
alter table customers
add column full_name varchar(100) not null,
drop column address ;

desc customers; (...)
```

Ejemplo 9: Opciones adicionales

```sql
create table customers (
   id int not null auto_increment,
   id_type varchar(3) not null,
   full_name varchar(100) null,
   primary key (id,id_type)
);

create table customers (
   id int not null auto_increment,
   id_type varchar(3) not null,
   full_name varchar(100) null,
   email varchar(100) not null unique,
   primary key (id,id_type)
);
```

```sql
create table customers (
   id int not null auto_increment,
   id_type varchar(3) not null default 'CC',
   full_name varchar(100) null,
   primary key (id,id_type)
);

create table customers (
   id int not null auto_increment,
   id_type varchar(3) not null default 'CC',
   full_name varchar(100) null,
   email varchar(100) not null unique,
   genre enum('F','M') not null,
   primary key (id,id_type)
);
```

INSERCCIÓN

Opción 1:

Especificar *<u>todos y cada uno</u>* de los campos

```
+------------+-------------+------+-----+---------+----------------+
| Field      | Type        | Null | Key | Default | Extra          |
+------------+-------------+------+-----+---------+----------------+
| id         | int(11)     | NO   | PRI | NULL    | auto_increment |
| first_name | varchar(30) | NO   |     | NULL    |                |
| last_name  | varchar(30) | YES  |     | NULL    |                |
| birth_date | date        | NO   |     | NULL    |                |
| genre      | varchar(15) | NO   |     | NULL    |                |
+------------+-------------+------+-----+---------+----------------+
5 rows in set (0,004 sec)
```

```
insert into singers values (21, 'MILEY', 'CYRUS', '1992-11-23', 'POP');
```

INSERCCIÓN

Opción 2:

Omitir auto-incremental, especificando los otros campos

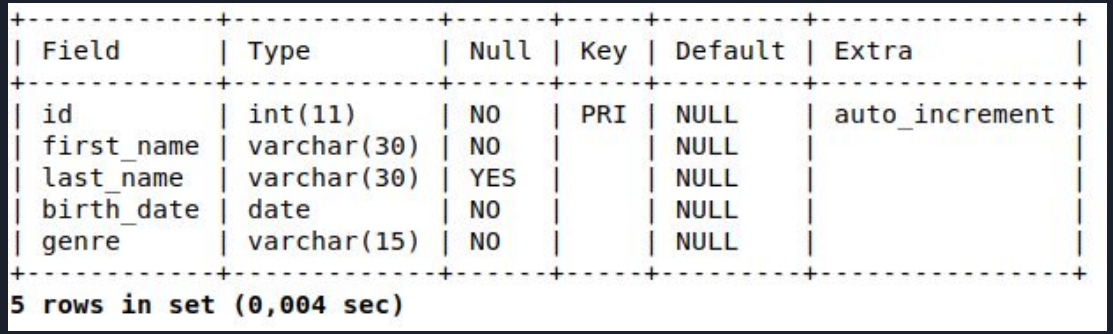
```
+------------+-------------+------+-----+---------+----------------+
| Field      | Type        | Null | Key | Default | Extra          |
+------------+-------------+------+-----+---------+----------------+
| id         | int(11)     | NO   | PRI | NULL    | auto_increment |
| first_name | varchar(30) | NO   |     | NULL    |                |
| last_name  | varchar(30) | YES  |     | NULL    |                |
| birth_date | date        | NO   |     | NULL    |                |
| genre      | varchar(15) | NO   |     | NULL    |                |
+------------+-------------+------+-----+---------+----------------+
5 rows in set (0,004 sec)
```

```
insert into singers (first_name, last_name, birth_date, genre)
values  ('MILEY', 'CYRUS', '1992-11-23', 'POP');
```

INSERCCIÓN

Opción 3:

Solo campos obligatorios, especificando los nombres



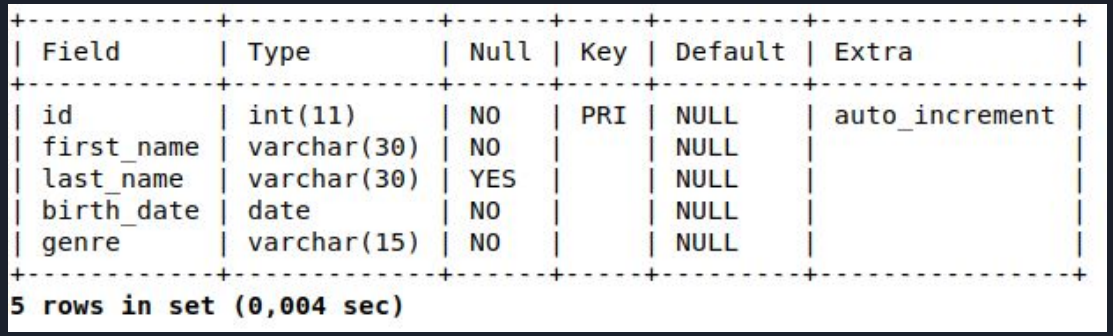
```
+------------+-------------+------+-----+---------+----------------+
| Field      | Type        | Null | Key | Default | Extra          |
+------------+-------------+------+-----+---------+----------------+
| id         | int(11)     | NO   | PRI | NULL    | auto_increment |
| first_name | varchar(30) | NO   |     | NULL    |                |
| last_name  | varchar(30) | YES  |     | NULL    |                |
| birth_date | date        | NO   |     | NULL    |                |
| genre      | varchar(15) | NO   |     | NULL    |                |
+------------+-------------+------+-----+---------+----------------+
5 rows in set (0,004 sec)
```

```
insert into singers (first_name, birth_date, genre)
values  ('MILEY', '1992-11-23', 'POP');
```

INSERCCIÓN

Opción 4:

Múltiples registros en una sola sentencia



Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	auto_increment
first_name	varchar(30)	NO		NULL	
last_name	varchar(30)	YES		NULL	
birth_date	date	NO		NULL	
genre	varchar(15)	NO		NULL	

5 rows in set (0,004 sec)

```
insert into singers (first_name, last_name, birth_date, genre) values
('MILEY', 'CYRUS', '1992-11-23', 'POP'),
('LADY', 'GAGA', '1986-03-28', 'POP'),
('RIHANNA', null, '1988-20-02', 'POP');
```

INSERCCIÓN

Opción 5:

Insertar los registros que son el resultado de una consulta

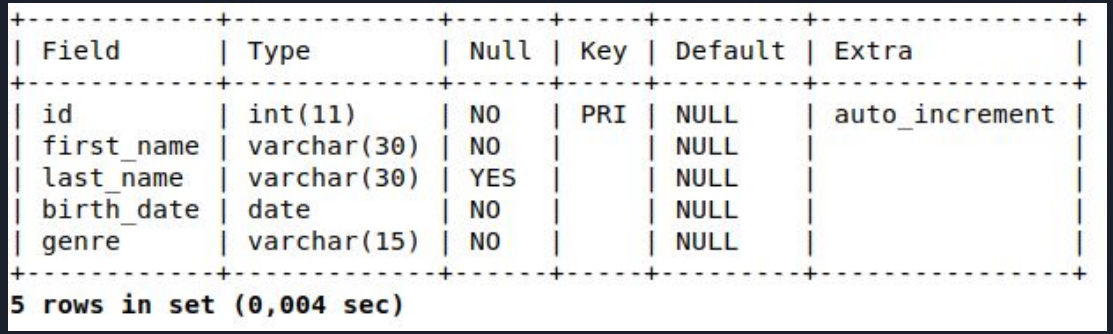
```
+------------+-------------+------+-----+---------+----------------+
| Field      | Type        | Null | Key | Default | Extra          |
+------------+-------------+------+-----+---------+----------------+
| id         | int(11)     | NO   | PRI | NULL    | auto_increment |
| first_name | varchar(30) | NO   |     | NULL    |                |
| last_name  | varchar(30) | YES  |     | NULL    |                |
| birth_date | date        | NO   |     | NULL    |                |
| genre      | varchar(15) | NO   |     | NULL    |                |
+------------+-------------+------+-----+---------+----------------+
5 rows in set (0,004 sec)
```

```
insert into singers (first_name, last_name, birth_date, genre)
select  first_name, last_name, birth_date, genre
from    another_singers
```

INSERCCIÓN

Opción 6:

Reemplazar campos si la clave primaria ya existe



Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	auto_increment
first_name	varchar(30)	NO		NULL	
last_name	varchar(30)	YES		NULL	
birth_date	date	NO		NULL	
genre	varchar(15)	NO		NULL	

5 rows in set (0,004 sec)

```
insert into singers (id, first_name, last_name, birth_date, genre)
values  (21, 'MILEY', '1992-11-23', 'POP')
on duplicate key update
    first_name = 'MILEY', last_name = 'CYRUS',
    birth_date = '1992-11-23', genre = 'POP';
```

INSERCCIÓN

Opción 7:

DDL + insert:
Crear tabla a partir del resultado de una consulta

```
+------------+-------------+------+-----+---------+----------------+
| Field      | Type        | Null | Key | Default | Extra          |
+------------+-------------+------+-----+---------+----------------+
| id         | int(11)     | NO   | PRI | NULL    | auto_increment |
| first_name | varchar(30) | NO   |     | NULL    |                |
| last_name  | varchar(30) | YES  |     | NULL    |                |
| birth_date | date        | NO   |     | NULL    |                |
| genre      | varchar(15) | NO   |     | NULL    |                |
+------------+-------------+------+-----+---------+----------------+
5 rows in set (0,004 sec)
```

```
create table singers2 as
select  *
from    singers
```

INSERCCIÓN

Opción 8, 9 ,10, 11……

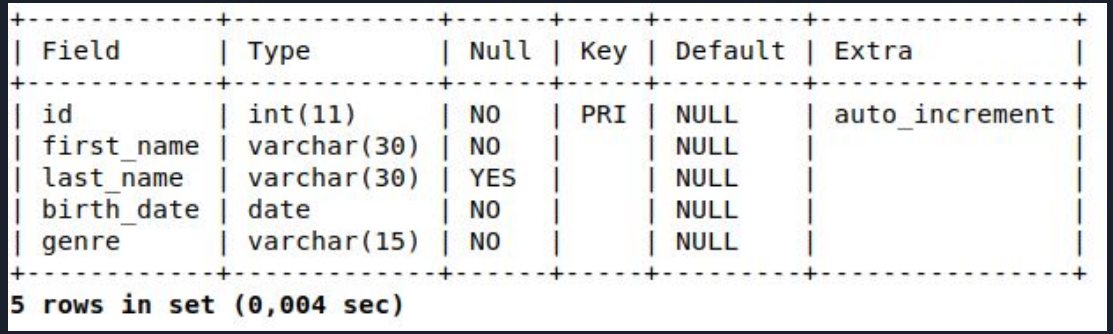
Las anteriores son sólo algunas de las formas básicas de ejecutar comandos tipo "insert".

Con el tiempo aprenderás a conocer más y a dominarlas!

ACTUALIZACIÓN

Opción 1:

Actualizar un campo de un registro especificando la clave primaria (la más común)



Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	auto_increment
first_name	varchar(30)	NO		NULL	
last_name	varchar(30)	YES		NULL	
birth_date	date	NO		NULL	
genre	varchar(15)	NO		NULL	

5 rows in set (0,004 sec)

```
update  singers
set     first_name = 'MILEY'
where   id = 21;
```

actualización

Opción 2:

Actualizar varios campos de un registro especificando la clave primaria

```
+------------+-------------+------+-----+---------+----------------+
| Field      | Type        | Null | Key | Default | Extra          |
+------------+-------------+------+-----+---------+----------------+
| id         | int(11)     | NO   | PRI | NULL    | auto_increment |
| first_name | varchar(30) | NO   |     | NULL    |                |
| last_name  | varchar(30) | YES  |     | NULL    |                |
| birth_date | date        | NO   |     | NULL    |                |
| genre      | varchar(15) | NO   |     | NULL    |                |
+------------+-------------+------+-----+---------+----------------+
5 rows in set (0,004 sec)
```

```
update  singers
set     first_name = 'MILEY',
        last_name = 'CYRUS'
where   id = 21;
```

ACTUALIZACIÓN

Opción 3:

Actualizar varios campos de varios registros especificando una condición

```
+------------+-------------+------+-----+---------+----------------+
| Field      | Type        | Null | Key | Default | Extra          |
+------------+-------------+------+-----+---------+----------------+
| id         | int(11)     | NO   | PRI | NULL    | auto_increment |
| first_name | varchar(30) | NO   |     | NULL    |                |
| last_name  | varchar(30) | YES  |     | NULL    |                |
| birth_date | date        | NO   |     | NULL    |                |
| genre      | varchar(15) | NO   |     | NULL    |                |
+------------+-------------+------+-----+---------+----------------+
5 rows in set (0,004 sec)
```

```
update  singers
set     genre = 'POPULAR'
where   genre = 'POP';
```

ACTUALIZACIÓN

Opción 4:

La más riesgosa (y una de las mejores formas de quedarse sin trabajo)

```
+------------+-------------+------+-----+---------+----------------+
| Field      | Type        | Null | Key | Default | Extra          |
+------------+-------------+------+-----+---------+----------------+
| id         | int(11)     | NO   | PRI | NULL    | auto_increment |
| first_name | varchar(30) | NO   |     | NULL    |                |
| last_name  | varchar(30) | YES  |     | NULL    |                |
| birth_date | date        | NO   |     | NULL    |                |
| genre      | varchar(15) | NO   |     | NULL    |                |
+------------+-------------+------+-----+---------+----------------+
5 rows in set (0,004 sec)
```

```
update  singers
set     genre = 'ROCK';
```



IMPORTANTE!!!

Repite conmigo:

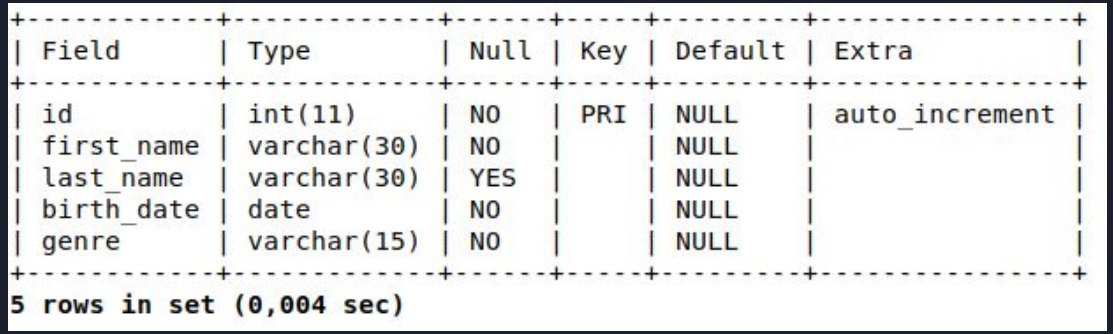
"No debo ejecutar un update sin where, salvo que sea estrictamente necesario!!!"



BORRADO

Opción 1:

Borrar de un registro especificando la clave primaria (la más común)



Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	auto_increment
first_name	varchar(30)	NO		NULL	
last_name	varchar(30)	YES		NULL	
birth_date	date	NO		NULL	
genre	varchar(15)	NO		NULL	

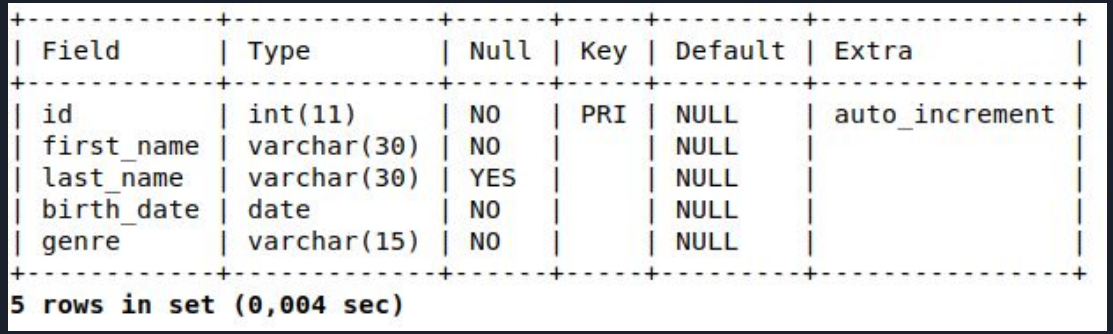
5 rows in set (0,004 sec)

```
delete
from    singers
where   id = 21;
```

BORRADO

Opción 2:

Borrar varios registros especificando una condición



Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	auto_increment
first_name	varchar(30)	NO		NULL	
last_name	varchar(30)	YES		NULL	
birth_date	date	NO		NULL	
genre	varchar(15)	NO		NULL	

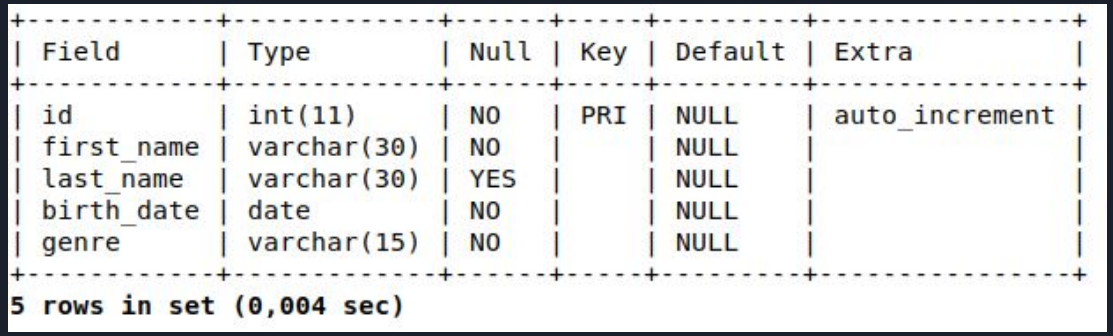
5 rows in set (0,004 sec)

```
delete
from    singers
where   genre = 'POP';
```

BORRADO

Opción 3:

Borrar varios registros especificando varias condiciones



Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	auto_increment
first_name	varchar(30)	NO		NULL	
last_name	varchar(30)	YES		NULL	
birth_date	date	NO		NULL	
genre	varchar(15)	NO		NULL	

5 rows in set (0,004 sec)

```
delete
from    singers
where   genre IN ('POP', 'ROCK')
and     birth_date >= '2001-01-01';
```

BORRADO

Opción 4:

La más riesgosa (y otra de las mejores formas de quedarse sin trabajo)

```
+------------+-------------+------+-----+---------+----------------+
| Field      | Type        | Null | Key | Default | Extra          |
+------------+-------------+------+-----+---------+----------------+
| id         | int(11)     | NO   | PRI | NULL    | auto_increment |
| first_name | varchar(30) | NO   |     | NULL    |                |
| last_name  | varchar(30) | YES  |     | NULL    |                |
| birth_date | date        | NO   |     | NULL    |                |
| genre      | varchar(15) | NO   |     | NULL    |                |
+------------+-------------+------+-----+---------+----------------+
5 rows in set (0,004 sec)
```

```
delete
from    singers;
```



IMPORTANTE!!!

Repite conmigo:

"No debo ejecutar un delete sin where, salvo que sea estrictamente necesario!!!"



BORRADO (Otra forma)

Opción única:

Eliminar completamente los datos de la tabla

NOTA: A nivel práctico, esta opción es equivalente a un delete sin where.

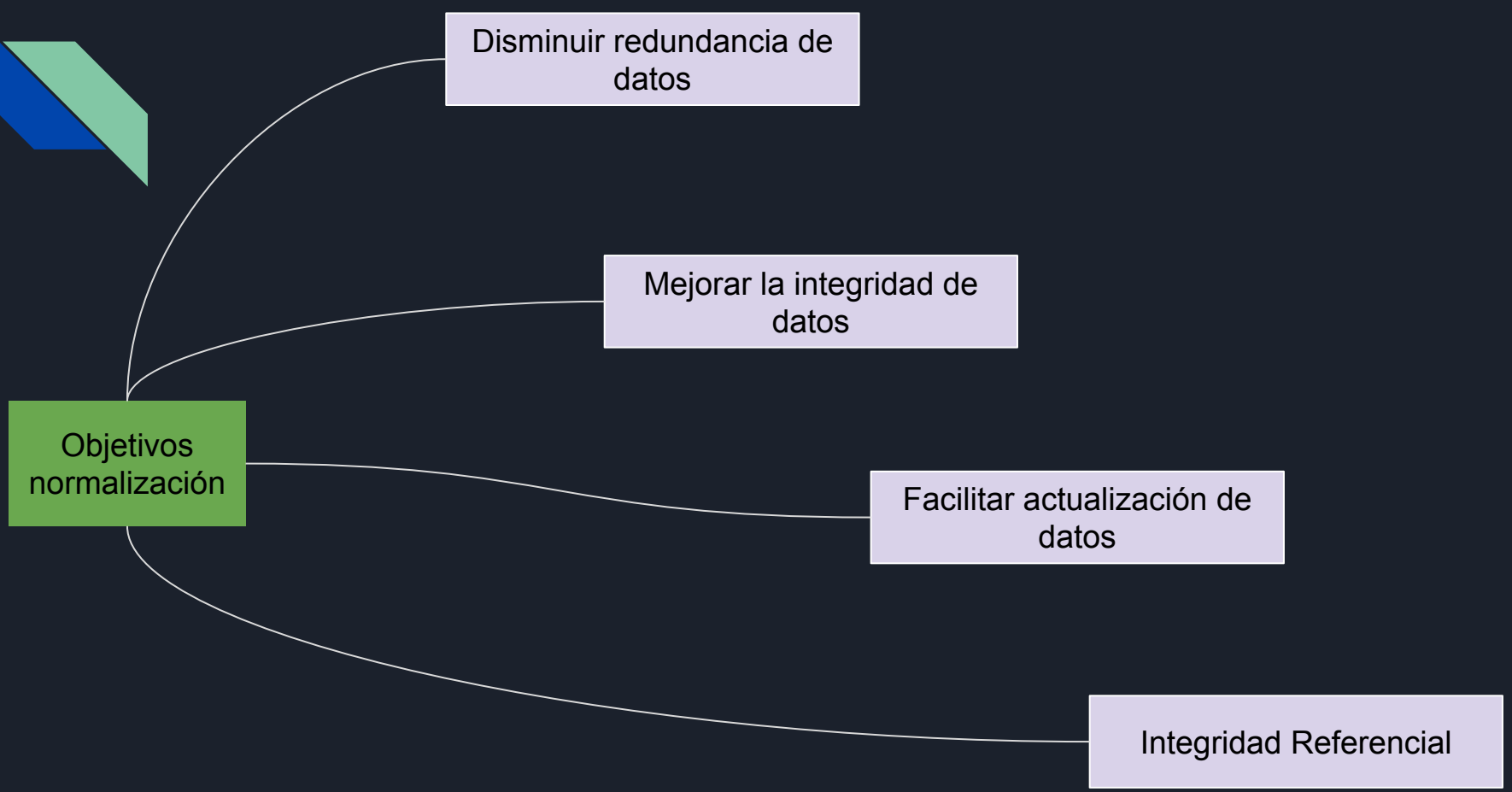
A nivel funcional es muy diferente. Truncate realmente hace un "drop" y luego un "create".

```
truncate table singers; //La tabla queda vacía y resetea auto incremental

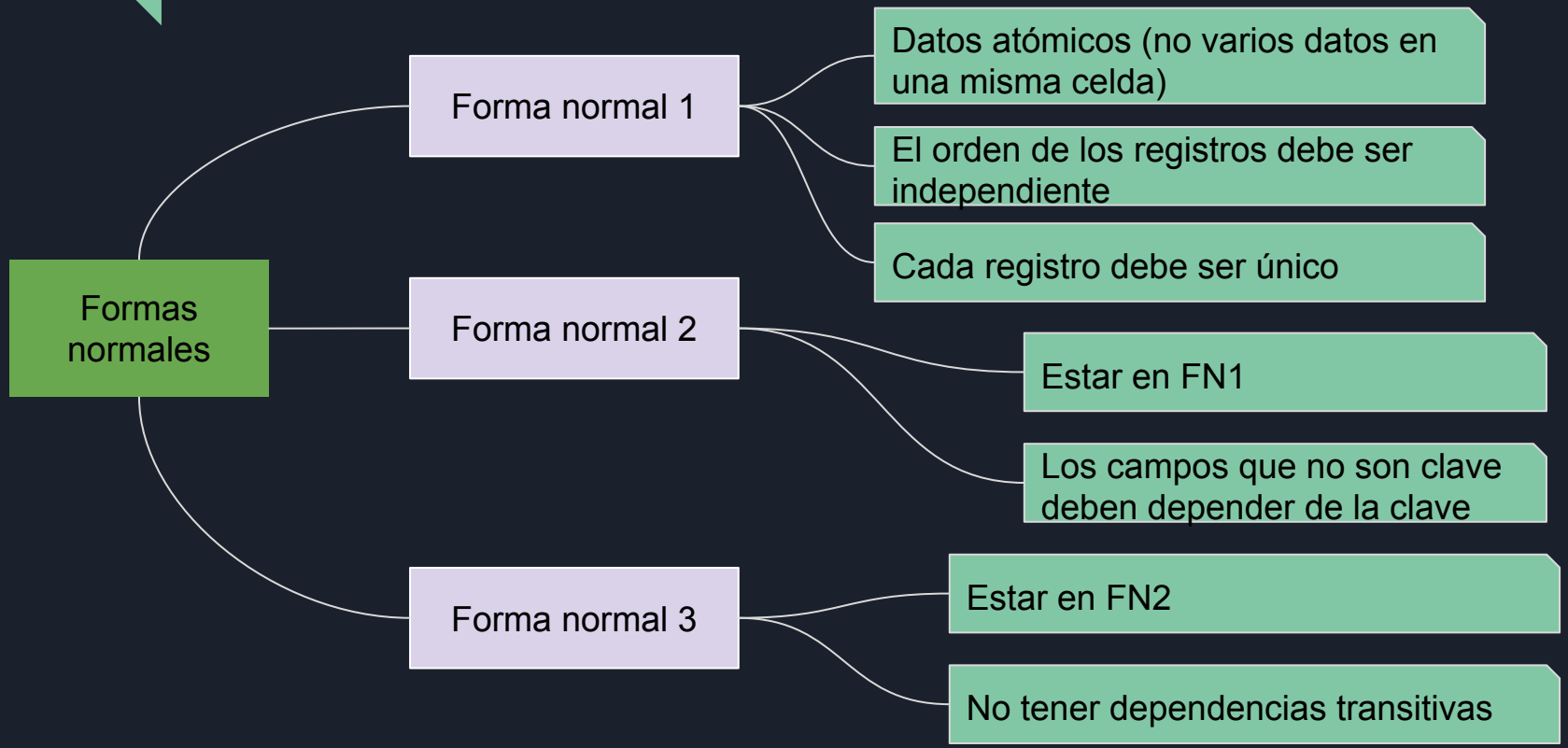
Truncate singers
where      id = 21;
// Esta sintaxis no está permitida ya que "truncate" no es un comando DML
```



Sección:
Normalización


Objetivos normalización
Disminuir redundancia de datos
Mejorar la integridad de datos
Facilitar actualización de datos
Integridad Referencial

Normalización: Reglas



Normalización: Dato adicional

Repite conmigo: "No debo guardar datos calculados en una base de datos" !!!

Cuando lo hayas hecho 100 veces, lee lo siguiente:

"He entendido que no debo guardar datos calculados en una base de datos, a no ser que sea estrictamente necesario" !!!

Sección:
CRUD

Que significa CRUD?

???	?????
???	?????
???	?????
???	?????

Que significa CRUD?

CREATE	Insertar datos en una tabla
READ	Consultar registros en una o varias tablas
UPDATE	Actualizar registros en una tabla
DELETE	Eliminar registros en una tabla

Operadores SQL

??????? (discusión verbal)

Funciones nativas

??????? (discusión verbal)

Subconsultas

??????? (discusión verbal)

Sección:
Relaciones (joins)

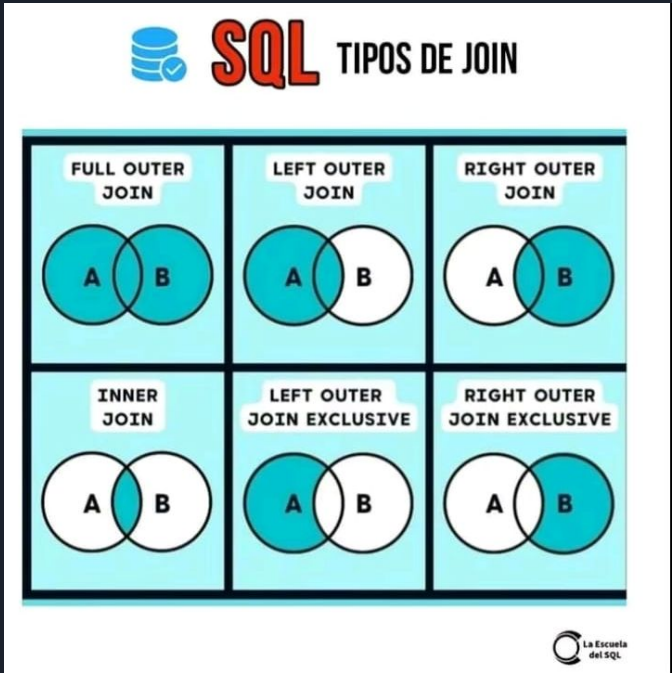
DEFINICIÓN: JOIN

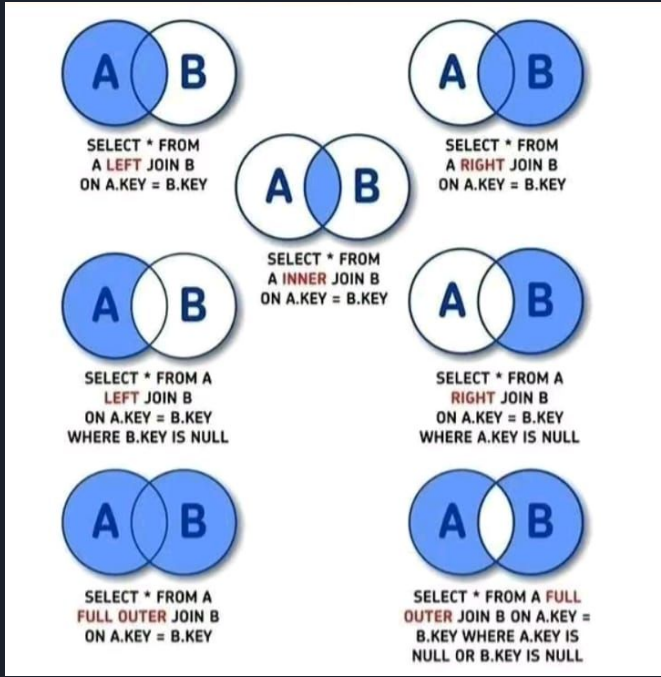
?????

DEFINICIÓN: JOIN

Join (o relación): Combinar registros de 2 (o más) tablas especificando una relación entre ellas. Generalmente es por clave foránea, pero puede ser por cualquier campo.

Joins - ejemplos





Fuente: https://www.tiktok.com/@laescueladelsql

Sección:
PL / SQL

Definición: PL/SQL

??????? (discusión verbal)

Definición: Función determinística

??????? (discusión verbal)

Definición: Función (almacenada)

??????? (discusión verbal)

Definición: Procedimiento (almacenado)

??????? (discusión verbal)

Función VS Procedimiento

??????? (discusión verbal)

Definición: Trigger (disparador)

??????? (discusión verbal)

Sección:
Administración

Definición: Usuario

??????? (discusión verbal)

Definición: Role (Rol)

??????? (discusión verbal)

Definición: Privilegio

??????? (discusión verbal)

Definición: Privilegio (asignación)

??????? (discusión verbal)

¿PREGUNTAS?

